

Лабораторная работа 3

Разработка приложения БД

Цель: Изучить основы проектирования приложений баз данных с использованием СУБД типа SQL. Освоить методику просмотра и редактирования таблиц в оконном и табличном интерфейсе, отбор записей таблиц

Часть 3. Работа с данными через табличный интерфейс

3.1 Просмотр и редактирование основной таблицы

Основной таблицей в учебной БД является *Книга поставок*. Данные для таблицы будем загружать из представления BookView.

```
CREATE VIEW BookView
AS
SELECT book_id, ndoc, dat, b.post_code, post_name,
       syrie, kol, cena,
       CAST(kol*cena as Numeric(10,2)) as Stoim
FROM Book as b join SPost as sp on b.post_code=sp.post_code
```

Представления используется вместо таблицы потому, что Книгу поставок необходимо соединить с таблицей Поставщики для отображения Наименования поставщика, а также выводить выражение из полей строки

Stoim = kol * cena

Форма просмотра основной таблицы БД (книги поставок Book) вызывается в пункте меню *Книга поставок* главной формы программного приложения.

Создаем форму просмотра основной таблицы Book, выбрав пункт меню

Проект – Добавить форму (Windows Forms)

Выберем шаблон создаваемой формы Windows Forms, зададим имя формы **FrmBook**.

На форму добавим таблицу **DataGridView**. Задаем элементу имя **dgBook**. В верхней части окна разместим элемент навигатора **BindingNavigator**. Задаем ему имя **bnBook**. Для полей фильтрации добавим панель между таблицей и навигатором (рис. 1).

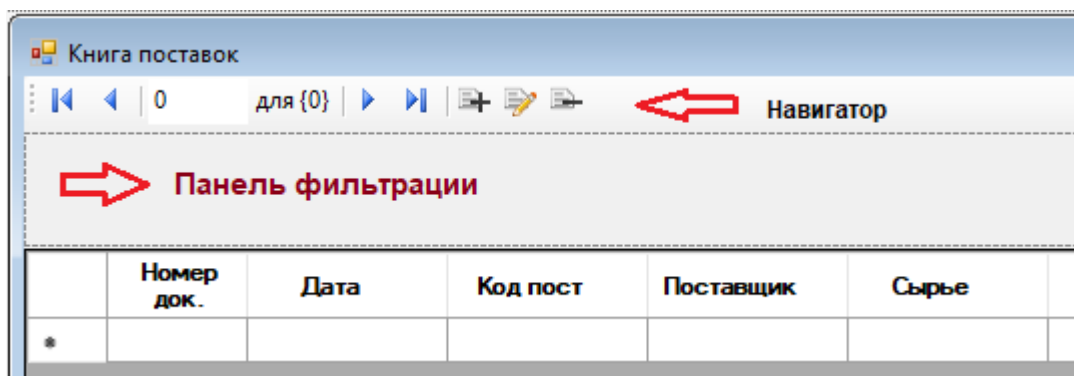


Рис. 1. Форма просмотра книги поставок

Зададим источник данных для табличного элемента **DataGridView**. Последовательность операций будет такой же как для справочника Поставщики (см.Лабораторная работа 3 ч.2).

Выполним другие настройки отображения данных таблицы **DataGridView**:

- Настроим стиль отображения заголовочной части таблицы (названия колонок). Для этого в списке элемента найдем свойство **ColumnHeadersDefaultCellStyle**
- колонку, связанную с колонкой таблицы БД **BookId**, сделаем невидимой (свойство **Visible=false**)
- Числовые значения (**Цена, Количество, Стоимость**) будем отображать с выравниванием по правой границе
- Стоимостные значения (**Цена, Стоимость**) будем выводить в формате с 2 знаками в дробной части. Для этого откроем редактор Правка столбцов и выберем свойство **DefaultCellStyle** (рис. 2).

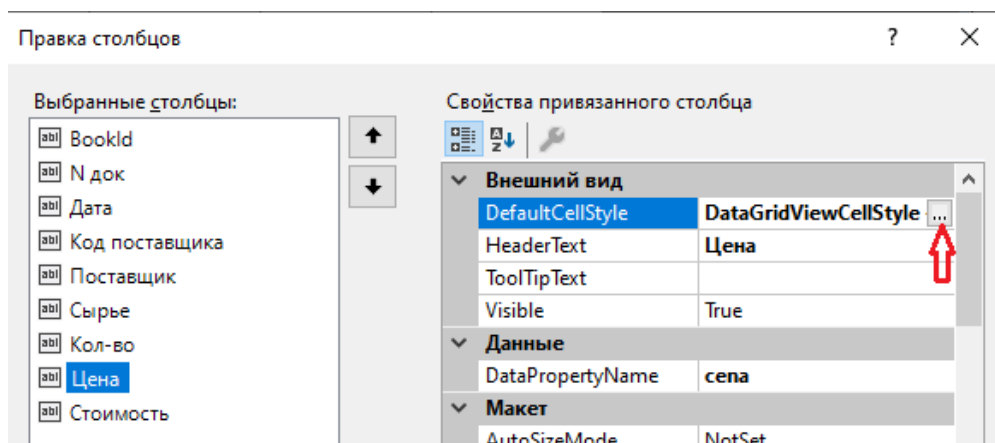


Рис. 2. Свойство настройки стиля отображения данных

Будем использовать подсвойства выравнивания **Alignment** и форматирования вывода **Format** (рис. 3)

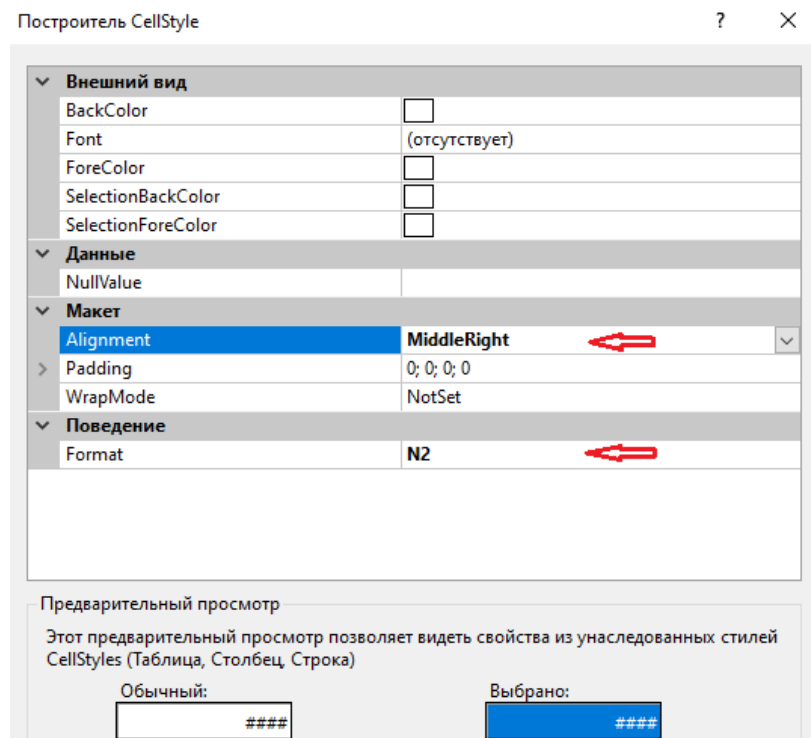


Рис. 3. Свойства выравнивания и формата отображения числовых данных

Переключимся в файл программного кода **FrmBook.cs**. В файле кода добавим поле:

```
private string strSql;
```

Загрузку данных в компонент таблицы привяжем к событию загрузки окна формы **Load**. В обработчике события окна **Load** добавим код:

```
private void FrmBook_Load(object sender, EventArgs e) {
    strSql = "select book_id,ndoc,dat,post_code,
               post_name,syrie,cena,kol,stoim
               from имя_схемы.bookView
               order by dat desc";
    loadData(strSql);
}
```

Вспомогательный метод **loadData** подгружает данные в табличный компонент при открытии формы и после выполнения операторов редактирования.

```
private void loadData(string strSql) {
    // Получение строки соединения из главной формы
    var con = FrmMain.connectionString;
    using (OdbcDataAdapter dadapter =
        new OdbcDataAdapter(strSql, con)) {
        // Загрузка таблицы
        DataTable table = new DataTable();
        dadapter.Fill(table);
        // Синхронизация табличного элемента и навигатора
        BindingSource bs = new BindingSource();
    }
```

```

        bs.DataSource = table.DefaultView;
        this.bnBook.BindingSource = bs;
        this.dgBook.DataSource = bs;
    }
}

```

Формирование и выполнение запрос для удаления строки справочника свяжем с событием Click кнопки удаления **sbDel**, размещенной на элементе навигатора. Обработчик события нажатия кнопки удаления строки таблицы:

```

private void sbDel_Click(object sender, EventArgs e){
    // Подтверждение удаления строки
    DialogResult dr = MessageBox.Show("Удалить строку?",
        "Confirmation", MessageBoxButtons.OKCancel,
        MessageBoxIcon.Question);
    if (dr != DialogResult.OK) return;
    // Определение индекса выделенной строки
    int ind = dgBook.CurrentCell.RowIndex;
    // Определение ключа (в первой колонке таблицы)
    string key = dgBook.Rows[ind].Cells[0].Value.ToString();
    // Формирование оператора SQL удаления строки
    string strSqlDel = "";
    strSqlDel += "DELETE FROM имя_схемы.book ";
    strSqlDel += " WHERE book_id=" + key;
    // Используем объект установленного соединения
    OdbcConnection con = FrmMain.odbcCon;
    OdbcCommand cmd = new OdbcCommand(strSqlDel, con);
    try {
        // Выполнение оператора SQL
        cmd.ExecuteNonQuery();
    }
    catch (Exception ex){
        MessageBox.Show("Ошибка удаления строки \n"+ex.Message);
        return;
    }
    // Обновление источника данных
    loadData(strSql);
    // Делаем текущей предыдущую строку
    if (ind > 0) ind--;
    dgBook.CurrentCell = dgBook[1, ind];
}

```

Программный код формирования и отправки запросов редактирование данных справочника рассмотрим после создание новой формы редактирования.

3.2 Создание полноэкранной формы редактирования

Добавим новую форму в проект и дадим ей имя (свойство **Name**) **FrmEdBook**. Зададим свойство расположения формы **StartPosition= CenterParent**. Добавим на форму элементы (рис. 4):

- 6 элемента статического текста **Label**

- 5 элементов полей ввода **TextBox** (**Name=edNdoc,edPostKod,edSyrie, edCena, edKol**)
- 1 элемент поля календаря **DateTimePicker** (**Name = edDat, Format=Short**)
- 1 элемент выпадающего списка **ComboBox** (**Name=cbPost**)
- 2 элемента кнопок **Button** (**Name=bOk, bCancel**)

Рис. 4. Форма редактирования

Переключимся в файл программного кода **FrmEdBook.cs**. В файле кода добавим поля:

```
private bool isNew = true; // Флаг режима добавления новой строки
private String old_key;    // Ключ (для режима редактирования)
```

Метод конструктора класса:

```
public FrmEdBook(bool isNew, String old_key)
{
    this.isNew = isNew;
    this.old_key = old_key;
    InitializeComponent();
}
```

Элемент выпадающего списка **ComboBox** используется для удобства заполнения поля внешнего ключа **Код поставщика** и поэтому список должен содержать данные справочника **Поставщики**.

Программный код загрузки данных разместим в обработчике события загрузки окна редактирования **Load**.

```
string strSql = "select post_code,post_name from spost";
var con = FrmMain.connectionString;
using (OdbcDataAdapter dadapter =
```

```

        new OdbcDataAdapter(strSql, con)) {
    DataTable table = new DataTable();
    dadapter.Fill(table);
    cbPost.DataSource = table;
    cbPost.DisplayMember = "post_name"; // столбец для отображения
    cbPost.ValueMember = "post_code";    // скрытый столбец ключа
}

```

Поле ввода кода поставщика и список поставщиков должны быть синхронизированы. Синхронизация означает, что при вводе **Кода поставщика** в списке должен автоматически выделяться связанное с данным кодом **Наименование поставщика**. И с другой стороны, при выборе в списке наименования поставщика в поле ввода автоматически должен устанавливаться его код. Эти действия привяжем к событиям поля ввода и поля выпадающего списка.

Для поля ввода синхронизацию свяжем с событием выхода из поля (потери фокуса) **Leave**

```

private void edPostKod_Leave(object sender, EventArgs e) {
    SynhronizeField();
}

```

Код вспомогательного метода **SynhronizeField**

```

private void edSynhronizeCombo() {
    int kod = System.Convert.ToInt16(edPostKod.Text);
    DataTable tbl = (DataTable)cbPost.DataSource;
    for (int i = 0; i < tbl.Rows.Count; i++)
        if (kod == System.Convert.ToInt16(tbl.Rows[i][0]))
            cbPost.SelectedIndex = i;
}

```

Для поля выпадающего списка синхронизацию свяжем с событием выбора элемента списка **SelectedIndexChanged**

```

private void cbPost_SelectedIndexChanged(object sender,
    EventArgs e) {
    int index = cbPost.SelectedIndex;
    DataTable tbl = (DataTable)cbPost.DataSource;
    edPostKod.Text = tbl.Rows[index][0].ToString();
}

```

Синхронизацию необходимо применить и при открытии окна. Поэтому полный код обработчика события загрузки окна **Load**:

```

private void FrmEdBook_Load(object sender, EventArgs e) {
    // Отключение обработчика ComboBox
    cbPost.SelectedIndexChanged -= cbPost_SelectedIndexChanged;
    // Текст оператора SQL загрузки данных для списка
    string strSql = "select post_code, post_name from spost";

    var con = FrmMain.connectionString;
    using (OdbcDataAdapter dadapter =
        new OdbcDataAdapter(strSql, con)) {
        DataTable table = new DataTable();
        dadapter.Fill(table);
    }
}

```

```

        cbPost.DataSource = table;
        cbPost.DisplayMember = "post_name"; // столбец для отображения
        cbPost.ValueMember = "post_code"; // столбец с кодом (ключом)
    }
    if (!isNew)
        // Синхронизация только в режиме редактирования
        edSynhronizeCombo();
    else
        cbPost.SelectedIndex = -1;
    // Подключение обработчика ComboBox
    cbPost.SelectedIndexChanged += cbPost_SelectedIndexChanged;
}

```

Добавим обработчики кнопок формы редактирования.

При нажатии кнопки **Отмена** необходимо просто закрыть форму. При этом в форму просмотра будет передан код возврата **DialogResult=Cancel**.

Программный код обработчика кнопки отмены изменений:

```

private void bCancel_Click(object sender, EventArgs e) {
    Close();
}

```

При нажатии кнопки **Сохранить** необходимо сформировать и отправить на сервер БД запрос с командой изменения данных. В режиме добавления на сервер будет послана команда INSERT, а в режиме редактирования – команда UPDATE. В случае успешного выполнения команды, форма закрывается, а в форму просмотра будет передан код возврата **DialogResult=OK**. При возникновении ошибки в ходе выполнения программы будет выведено сообщение об ошибке и форма редактирования закрыта не будет.

```

private void bOk_Click(object sender, EventArgs e) {
    string strSql = "";
    if (isNew) {
        strSql += "INSERT INTO имя_схемы.book(ndoc,dat,post_code,syrie,kol,cena)";
        strSql += values("";
        strSql += edNdoc.Text;
        strSql += ", '" + edDat.Value.ToString("yyyy-MM-dd") + "'";
        strSql += ", " + edPostKod.Text;
        strSql += ", '" + edSyrie.Text + "'";
        strSql += ", " + edKol.Text;
        strSql += ", " + edCena.Text + ")";
    }
    else {
        strSql += "UPDATE имя_схемы.book SET ";
        strSql += "ndoc=" + edNdoc.Text;
        strSql += ",post_code=" + edPostKod.Text;
        strSql += ",syrie='" + edSyrie.Text + "'";
        strSql += ",dat='" + edDat.Value.ToString("yyyy-MM-dd") + "'";
        strSql += ",kol='" + edKol.Text + "'";
        strSql += ",cena='" + edCena.Text + "'";
        strSql += " WHERE book_id=" + old_key;
    }
    String conStr = FrmMain.connectionString;
    OdbcConnection con = FrmMain.odbcCon;
    OdbcCommand cmd = new OdbcCommand(strSql, con);
    try {

```

```

        cmd.ExecuteNonQuery();
        this.DialogResult = DialogResult.OK;
        Close();
    } catch (Exception ex) {
        MessageBox.Show("Ошибка добавления/изменения строки \n" +
            ex.Message);
        return;
    }
}

```

Обратите внимание на вставку в оператор SQL значения даты. Значение даты преобразуется в формат представления даты в БД **"yyyy-mm-dd"**.

Вернемся в форму просмотра **FrmBook** и добавим программный код для кнопок редактирования.

Программный код обработчика кнопки вызова формы редактирования в режиме добавления новой строки (нового поставщика):

```

private void sbAdd_Click(object sender, EventArgs e){
    FrmEdBook frm = new FrmEdSBook(true, "");
    // Заголовок формы в режиме добавления нового поставщика
    frm.Text = "Добавление поставки";
    frm.ShowDialog();
    // Перегружаем данные в случае добавления новой строки
    if (frm.DialogResult == DialogResult.OK){
        loadData(strSql);
    }
    frm.Dispose();
}

```

Программный код обработчика кнопки вызова формы редактирования окна в режиме редактирования существующей строки:

```

private void sbEdit_Click(object sender, EventArgs e) {
    // Запоминаем номер текущей строки
    int ind = dgBook.CurrentRow.Index;
    // Создаем форму (второй параметр – ключ редактируемой строки)
    FrmEdBook frm = new FrmEdBook(false,
        dgBook.Rows[ind].Cells[0].Value.ToString());
    // Заголовок формы в режиме редактирования
    frm.Text = "Редактирование поставщика";
    // Заполняем поля ввода текущими значениями
    frm.edNdoc.Text = dgBook.Rows[ind].Cells[1].Value.ToString();
    frm.edDat.Text =
        dgBook.Rows[ind].Cells[2].Value.ToString();
    frm.edPostKod.Text =
        dgBook.Rows[ind].Cells[3].Value.ToString();
    frm.edSyrie.Text =
        dgBook.Rows[ind].Cells[5].Value.ToString();
    // Строковое представление чисел в региональном формате
    // 3193.23:
    // en-US: 3193.23
    // ru-RU: 3193,23
    System.Globalization.CultureInfo culture =
        new System.Globalization.CultureInfo("en-US");
    frm.edCena.Text = Convert.ToString(System.Convert.ToDecimal
        (dgBook.Rows[ind].Cells[6].Value), culture);
}

```

```

frm.edKol.Text = Convert.ToString(System.Convert.ToDecimal
                                (dgBook.Rows[ind].Cells[7].Value), culture);
frm.ShowDialog();
// Перегружаем данные в случае сохранения изменений
if (frm.DialogResult == DialogResult.OK) {
    loadData(strSql);
    dgBook.CurrentCell= dgBook[0,ind];
}
frm.Dispose();
}

```

Обратите внимание, как берутся значения чисел с дробной частью. Эти значения для редактирования будем переводить в строковое представление чисел в региональном формате **en-US** (с разделителем целой и дробной части числа «точкой»).

3.3 Реализация отбора данных

Часто при работе с данными желательно видеть не все записи таблицы БД, а лишь некоторую их часть, соответствующую некоторым условиям.

Для наборов данных, реализованных компонентами запросами, следует сформировать запрос с заданными условиями, которые указывают в предикатах раздела WHERE.

Добавим на панель фильтрации (между таблицей и навигатором) элементы для отбора данных (рис. 5):

- 2 поля-календаря **DateTimePicker** для задания диапазона дат (**Name: edDat1, edDat2**);
- 2 поля ввода для задания диапазона номеров документов (**Name: edNd1, edNd2**);
- поле ввода для задания названия сырья (**Name: edSyr**);
- поле ввода для задания названия поставщика (**Name: edPost**);
- 2 поля ввода для задания диапазона цен (**Name: edPr1, edPr2**);
- 2 поля ввода для задания диапазона значений количества (**Name: edKol1, edKol2**);
- кнопку установки фильтра (**Name: bFilter**);
- кнопку очистки фильтра (**Name: bClear**).

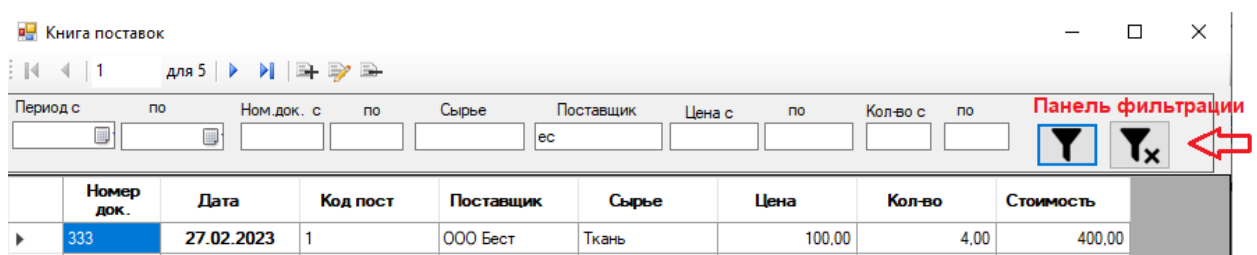


Рис. 5 Панель фильтрации

Есть особенности настройки полей для задания дат. По умолчанию в этих полях будет выведена текущая (системная) дата. Желательно при открытии формы очистить эти поля. Для этого надо установить для элемента-календаря минимально возможную дату **DateTimePicker.MinimumDateTime** и для этой даты задать формат отображения «пустая строка».

```

dateTimePicker.Format = DateTimePickerFormat.Custom;
dateTimePicker.CustomFormat = " ";

```

Создадим метод очистки поля-календаря (установки даты в минимально возможную величину):

```
private void Clear_DateTimePicker() {
    edDat1.Value = DateTimePicker.MinimumDateTime;
    edDat2.Value = DateTimePicker.MinimumDateTime;
}
```

Для полей-календарей зададим обработчик события изменения значения **ValueChanged**

```
private void DateTimePicker_ValueChanged(object sender,
    EventArgs e) {
    DateTimePicker obj = (DateTimePicker)sender;
    if (obj.Value == DateTimePicker.MinimumDateTime) {
        // Устанавливаем для возможности позиционирования на текущую дату
        // при нажатии на кнопку выбора (раскрытия календаря)
        obj.Value = DateTime.Now;
        obj.Format = DateTimePickerFormat.Custom;
        obj.CustomFormat = " ";
    } else {
        obj.Format = DateTimePickerFormat.Short;
    }
}
```

Добавим обработчики для события **Click** кнопок.

При нажатии на кнопку очистки фильтра будем очищать поля задания значений и восстанавливать исходный запрос получения данных

```
private void bClear_Click(object sender, EventArgs e) {
    Clear_DateTimePicker();
    edNd1.Text = string.Empty;
    edNd2.Text = string.Empty;
    edPost.Text = string.Empty;
    edSyr.Text = string.Empty;
    edPr1.Text = string.Empty;
    edPr2.Text = string.Empty;
    edKol1.Text = string.Empty;
    edKol2.Text = string.Empty;
    loadData(strSql);
}
```

При нажатии на кнопку установки фильтра будем формировать условие отбора для заданных значений и это условие будем использовать для отбора данных в разделе оператора запроса **WHERE**. Для упрощения кода в начало условного выражения запишем тождественно истинное условие «1=1»

```
private void bFiltr_Click(object sender, EventArgs e) {
    string where = "";
    string str;
    if (edDat1.Text != " ") {
        where += " and dat>='" + edDat1.Value.ToString("yyyy-MM-dd")+"'";
    }
    if (edDat2.Text != " ") {
        where += " and dat<='" + edDat2.Value.ToString("yyyy-MM-dd")+"'";
    }
}
```

```

    }
    if (edNd1.Text != string.Empty){
        where += " and ndoc>= " + edNd1.Text;
    }
    if (edNd2.Text != string.Empty){
        where += " and ndoc<= " + edNd2.Text;
    }
    if (edSyr.Text != string.Empty){
        where += " and Syrie ilike '%" +
            edSyr.Text + "%'";
    }
    if (edPost.Text != string.Empty){
        where += "and post_name "
            + " ilike '%"
            + edPost.Text + "%'";
    }
    if (edPr1.Text != string.Empty){
        where += " and cena>= " + edPr1.Text;
    }
    if (edPr2.Text != string.Empty){
        where += " and cena<= " + edPr2.Text;
    }
    if (edKol1.Text != string.Empty){
        where += " and kol>= " + edKol1.Text;
    }
    if (edKol2.Text != string.Empty){
        where += " and kol<= " + edKol2.Text;
    }

    if (where == "")
        str = strSql;
    else
        str = "SELECT book_id,ndoc,dat,post_code,
            post_name,syrie,cena,kol,stoim
            from имя_схемы.bookView WHERE 1=1 "
            + where + " ORDER BY dat desc";

    loadData(str);
}

```

Задание:

Дополнить программное приложение:

- 1) формами просмотра и редактирования основной таблицы
- 2) функционалом отбора данных основной таблицы по заданным критериям (по колонкам с числовым, хронологическим и символьным значениями, диапазону значений колонки, колонке справочной таблицы)

Содержание отчета:

- 1) схема БД;
- 2) текст представления соединения таблиц;
- 3) программный код метода загрузки данных;
- 4) программный код метода удаления строки;
- 5) программный код формы редактирования основной таблицы;
- 6) скриншот формы просмотра основной таблицы с заполненными данными;
- 7) скриншот формы редактирования основной таблицы с заполненными данными;
- 8) скриншот результатов отбора данных (панель отбора и отобранные строки).